

Actividad Autónoma N.º 09

Diferencias por plataforma y evidencia en Android

Proyecto	PharmaMobile sobre PharmaSoft
Estudiante	Luciana Gutierrez
Asignatura	Desarrollo de Aplicaciones Móviles
Ciclo y semestre	VI 2026-2
Sesión	Actividad Autónoma N.º 09
Fecha	8 de octubre de 2026
Rama	feature/expect-actual-Gutierrez

Resultado

Se implementó una tercera capacidad nativa con expect/actual, se conectó a la interfaz, se verificó Android 17 y se validó la compilación Kotlin de los targets iOS. Las evidencias iOS no se realizaron porque no se dispone de Mac, Xcode ni simulador iOS.

1. Estado inicial y alcance

La Guía Práctica 09 ya había incorporado dos capacidades: el formato monetario mediante expect/actual y el uso de un Compartidor con implementaciones nativas registradas por Koin. Esta actividad no rehace esas funciones; las inventaría, explica sus diferencias y añade Información del dispositivo como tercera capacidad.

1.1 Capacidades heredadas de la práctica

Capacidad	Estado inicial	Uso actual
formatearSoles	Implementada en Android e iOS	ProductoUi muestra precios nativos
Compartidor	Implementado e inyectado	ProductoViewModel abre la UI nativa
platformModule	Implementado por plataforma	Motores Ktor, URL y Compartidor

1.2 Trabajo realizado en la actividad

- Inventario de capacidades con firmas, source sets, rutas y APIs reales.
- Informe comparativo de 745 palabras basado en el código del proyecto.
- InfoDispositivo con expect, actual Android y actual iOS.
- Pantalla Acerca de integrada a la navegación Compose.
- Prueba real de compilación con un actual ausente y restauración inmediata.
- README actualizado y cuatro commits propios sin push.

1.3 Restricción de evidencias

Las evidencias Android se obtuvieron directamente del emulador. No se incluyen capturas iOS: no se dispone de Mac/Xcode y no se reemplazaron por imágenes fabricadas. El código iOS sí fue compilado por las tareas Kotlin de shared, pero no enlazado como framework ni ejecutado.

2. Producto 1. Inventario de capacidades

El inventario incluye todos los mecanismos comunes relevantes encontrados en el proyecto y la nueva capacidad implementada para esta actividad.

Capacidad	Firma común	Archivo commonMain
Formato de moneda	expect fun formatearSoles(valor: Double): String	platform/Formato.kt
Compartir	interface Compartidor { fun compartir(texto: String) }	domain/platform/Compartidor.kt
Módulo DI	expect val platformModule: Module	di/AppModule.kt
Plataforma base	expect fun getPlatform(): Platform	Platform.kt
Información del dispositivo	expect class InfoDispositivo()	platform/InfoDispositivo.kt

Capacidad	Android y API	iOS y API
Formato	Formato.android.kt · NumberFormat y Locale	Formato.ios.kt · NSNumberFormatter y NSLocale
Compartir	CompartidorAndroid.kt · Context e Intent	CompartidorIos.kt · UIActivityViewController
Módulo DI	PlatformModule.android.kt · Koin y androidContext	PlatformModule.ios.kt · Koin y Darwin
Plataforma base	Platform.android.kt · Build.VERSION.SDK_INT	Platform.ios.kt · UIDevice
Información	InfoDispositivo.android.kt · Build.VERSION.RELEASE	InfoDispositivo.ios.kt · UIDevice

2.1 Rutas base

Todos los archivos Kotlin se encuentran bajo `shared/src/<sourceSet>/kotlin/pe/edu/upeu/pharmamobilee/`. La tabla usa rutas relativas a este paquete para conservar legibilidad.

3. Producto 2. Informe comparativo

Pregunta 1. Por qué difiere el formato de moneda

Las dos implementaciones reciben el mismo Double y buscan representar soles con configuración peruana, pero se ejecutan sobre bibliotecas diferentes. En Android, Formato.android.kt llama a `NumberFormat.getCurrencyInstance(Locale("es", "PE"))`; es la API de internacionalización de la JVM y aplica las reglas disponibles en el runtime de Java del dispositivo. En iOS, Formato.ios.kt crea un `NSNumberFormatter`, le asigna `NSNumberFormatterCurrencyStyle` y `NSLocale("es_PE")`, y convierte el Double a `NSNumber`; esas clases pertenecen a Foundation. El locale expresa la intención cultural, pero no obliga a JVM y Foundation a producir exactamente la misma secuencia de espacios o símbolos porque cada framework aporta sus propios datos y convenciones. En la ejecución Android real, Paracetamol 500 mg se mostró como S/ 4.50. El código iOS está escrito y compilado, pero no se ejecutó; por ello no se afirma su resultado literal.

Pregunta 2. Context en Android y UIKit en iOS

CompartidorAndroid recibe un Context porque iniciar una actividad pertenece al modelo de componentes Android. Construye un `Intent.ACTION_SEND`, configura `text/plain`, coloca el mensaje en `Intent.EXTRA_TEXT` y crea el selector. Como Koin entrega el contexto de aplicación mediante `androidContext()`, no existe una Activity receptora directa; por eso incorpora `FLAG_ACTIVITY_NEW_TASK` antes de `context.startActivity`. Compartidorlos no recibe un objeto equivalente: UIKit expone `UIApplication.sharedApplication` y desde su controlador raíz presenta un `UIActivityViewController`. `PlatformModule.android` crea `CompartidorAndroid(androidContext())`, mientras `PlatformModule.ios` registra `Compartidorlos()` sin argumentos. `ProductoViewModel` depende solo de la interfaz común.

Pregunta 3. expect actual frente a interfaz e inyección

Si `formatearSoles` fuera una interfaz, habría que definir un contrato, crear implementaciones, registrarlas en Koin e inyectar una instancia en cada consumidor. La estrategia mejora la sustitución por dobles y es apropiada cuando existe estado, dependencias o efectos. Para una función pura añade código ceremonial y acoplamiento al contenedor. Con `expect/actual`, el compilador selecciona la implementación del source set y comprueba firma y paquete; la llamada común permanece directa. La desventaja es que sustituirla en pruebas es menos flexible y la falta de un actual bloquea el target. PharmaMobile usa ambas estrategias con intención: `formatearSoles` es determinista y adecuado para `expect/actual`; `Compartidor` produce un efecto nativo, requiere Context en Android y se beneficia de interfaz e inyección. Así el `ViewModel` admite un compartidor falso sin abrir UI nativa.

3. Informe comparativo continuación

Pregunta 4. Qué ocurre si falta un actual

La prueba se ejecutó comentando temporalmente la función de Formato.android.kt y lanzando `.\gradlew.bat :shared:compileAndroidMain --console=plain`. Kotlin detuvo la compilación antes de generar el módulo Android. El mensaje literal fue el siguiente:

```
e: file:///C:/Users/lucia/AndroidStudioProjects/PHARMAMOBILEE/shared/src/commonMain/kotlin/pe/edu/upeu/pharmamobilee/platform/Formato.kt:3:1 Expected formatearSoles has no actual declaration in module <commonMain> for JVM
```

Después de registrar el resultado se restauró inmediatamente el actual. Esto demuestra que cada target que compila el código común debe suministrar una declaración compatible en paquete, nombre, parámetros y retorno.

Pregunta 5. Qué debe permanecer en commonMain

ProductoViewModel no debería bajar a androidMain ni iosMain. Coordina casos de uso, actualiza ProductoUiState, procesa Result, conserva el listado durante las mutaciones, distribuye errores por campo y recarga productos después del CRUD. Ninguna regla depende de Context, Intent, UIKit ni Foundation. Duplicarlo introduciría dos flujos susceptibles de divergir, repetiría pruebas y podría producir transiciones distintas para la misma respuesta de PharmaSoft. El límite correcto es el existente: el ViewModel permanece en commonMain y recibe contratos como Compartidor; solo la acción que abre la interfaz nativa vive en el source set específico. El mismo criterio mantiene en común el repositorio REST, los casos de uso y las validaciones.

Extensión del informe

El contenido de las cinco respuestas tiene 745 palabras en la documentación fuente, dentro del intervalo solicitado de 600 a 900 palabras.

4. Producto 3. Información del dispositivo

4.1 Contrato común

```
package pe.edu.upeu.pharmamobilee.platform

expect class InfoDispositivo() {
    val sistema: String
    val version: String
}
```

4.2 Implementación Android

```
actual class InfoDispositivo actual constructor() {
    actual val sistema: String = "Android"
    actual val version: String = Build.VERSION.RELEASE
}
```

4.3 Implementación iOS

```
actual class InfoDispositivo actual constructor() {
    actual val sistema: String = UIDevice.currentDevice.systemName()
    actual val version: String = UIDevice.currentDevice.systemVersion
}
```

4.4 Integración con Compose

AcercaDeScreen crea el contrato común con remember, muestra Sistema y Versión, y se abre desde la nueva opción Acerca de del menú. El composable no importa android.* ni platform.*.

4.5 Aislamiento

Comando ejecutado:

```
rg -n '^import\s+(android\.|platform\.)' shared/src/commonMain
```

Resultado real: 0 coincidencias.

5. Producto 4. Evidencias Android

Las tres imágenes fueron obtenidas de la aplicación Android correspondiente a la misma rama. No se añadieron evidencias iOS.

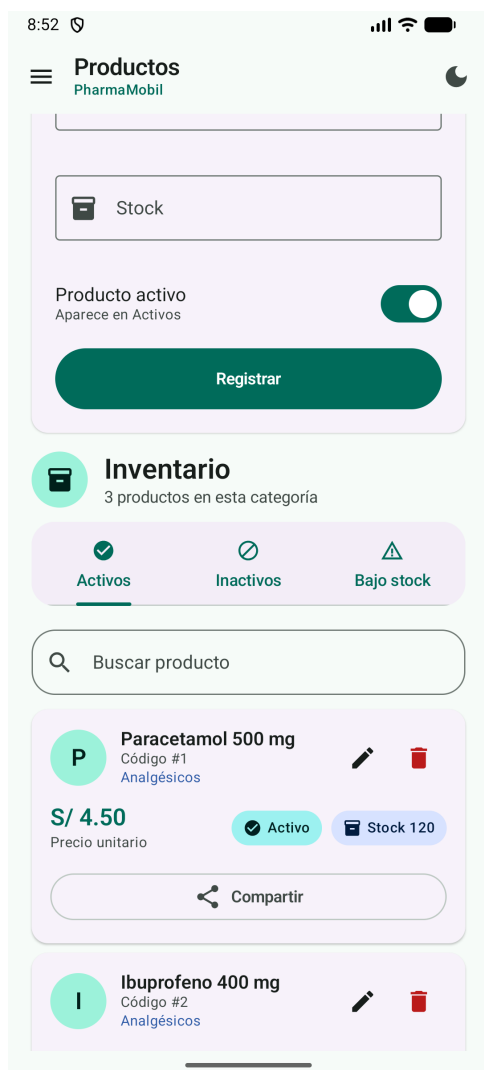


Figura 1. Precio Android real: Paracetamol 500 mg, S/ 4.50.

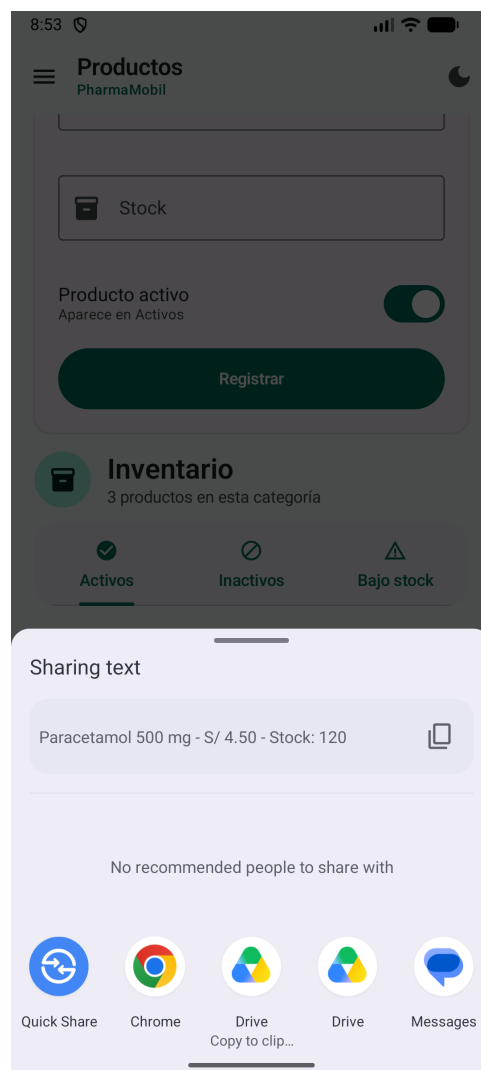


Figura 2. Selector nativo con el texto del producto visible.

5.1 Información del dispositivo en Android

El APK compilado se instaló en el emulador Medium_Phone. La pantalla Acerca de consumió el actual Android y mostró el sistema y la versión devueltos por Build.VERSION.RELEASE.

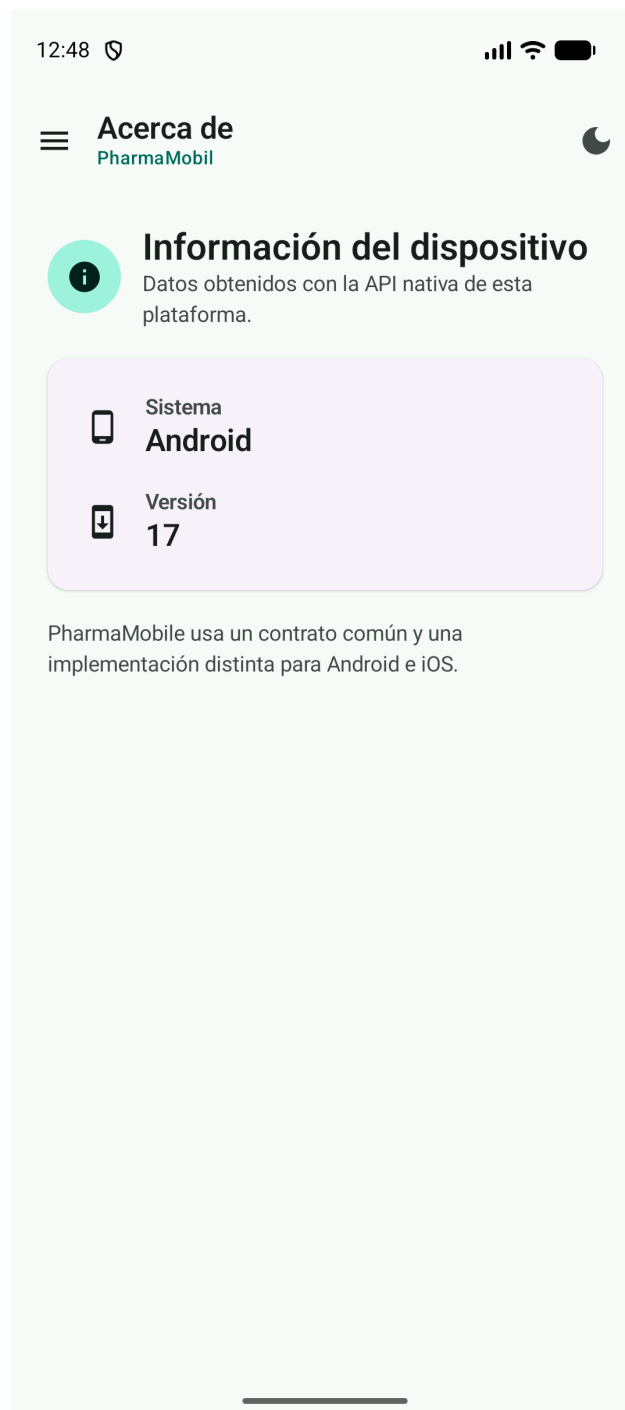


Figura 3. InfoDispositivo ejecutado en Android 17.

5.2 Evidencia del compilador

La herramienta visual de Android Studio no estuvo disponible para automatizar una captura del panel Build. Para no fabricar una imagen, se conserva el registro textual real producido por la tarea de Gradle.

```
Comando:
.\gradlew.bat :shared:compileAndroidMain --console=plain

> Task :shared:compileAndroidMain FAILED
e: file:///C:/Users/lucia/AndroidStudioProjects/PHARMAMOBILEE/shared/src/commonMain/kotlin/pe/edu/upeu/pharm
amobilee/platform/Formato.kt:3:1 Expected formatearSoles has no actual declaration in module <commonMain>
for JVM

BUILD FAILED in 28s
13 actionable tasks: 8 executed, 5 up-to-date
```

El actual Android fue restaurado inmediatamente. Luego :shared:compileAndroidMain, las pruebas y la compilación del APK finalizaron correctamente.

6. Verificación técnica y repositorio

Verificación	Resultado real
:shared:compileAndroidMain	BUILD SUCCESSFUL
:shared:assemble	BUILD SUCCESSFUL en 2 min 10 s
compileKotlinlosArm64	Compilación Kotlin correcta
compileKotlinlosSimulatorArm64	Compilación Kotlin correcta
Enlace y ejecución iOS	No realizados en Windows
:shared:testAndroidHostTest	34 pruebas, 0 fallos, 0 errores, 0 omitidas
:androidApp:assembleDebug	BUILD SUCCESSFUL
Ejecución Android	Medium_Phone · Android 17

6.1 Commits de la actividad

Hash	Mensaje
e0d3a08	docs: registra inventario de capacidades nativas
a1bfc75	docs: prepara comparacion de implementaciones por plataforma
2599ce6	feat: agrega informacion del dispositivo con expect actual
a22e5b0	docs: actualiza codigo especifico de plataforma

6.2 Rama

<https://github.com/LucianaGutierrez-049/PharmaMobile/tree/feature/expect-actual-Gutierrez>

Estado al cerrar este informe: los commits permanecen locales y no se hizo push.

7. Estado de evidencias iOS

No se realizaron capturas ni ejecución iOS porque no se dispone de Mac, Xcode, dispositivo ni simulador iOS. Esta ausencia se declara expresamente y no se substituyó con imágenes generadas.

Evidencia solicitada	Estado
Precio con NSNumberFormatter	NO REALIZADA · requiere Mac/Xcode
UIActivityViewController	NO REALIZADA · requiere Mac/Xcode
InfoDispositivo con UIDevice	NO REALIZADA · requiere Mac/Xcode

7.1 Qué sí se verificó del código iOS

La tarea :shared:assemble ejecutó compileKotlinlosArm64 y compileKotlinlosSimulatorArm64 sin errores de código. Las tareas de enlace de frameworks fueron omitidas en Windows; por tanto, esta comprobación no se presenta como ejecución de la aplicación.

8. Checklist final

N.º	Criterio	Estado
1	PDF nombrado S09_ActividadAutonoma_Gutierrez.pdf	CUMPLE
2	Portada con nombre, sesión, fecha y rama	CUMPLE
3	Inventario con firmas, source sets, archivos y APIs	CUMPLE
4	Cinco preguntas basadas en el código propio	CUMPLE
5	Mensaje literal del compilador	CUMPLE
6	Tercera capacidad con expect y dos actual	CUMPLE
7	Tercera capacidad conectada a la interfaz	CUMPLE
8	Sin imports nativos en commonMain	CUMPLE
9	Capturas de Android e iOS	PARCIAL · Android sí, iOS no disponible
10	README con Código específico de plataforma	CUMPLE
11	Commits identificados en la rama propia	CUMPLE

8.1 Evaluación transparente

La implementación, el aislamiento, el inventario, el análisis y Android están completos. La rúbrica oficial exige ejecución en ambas plataformas para el nivel máximo en tercera capacidad y evidencias. Debido a la decisión de entregar únicamente Android, el informe no declara 20/20 ni afirma una ejecución iOS inexistente.

9. Conclusión

PharmaMobile distingue correctamente el código común del específico de plataforma. El formato monetario usa expect/actual; Compartidor usa interfaz e inyección por tratarse de un efecto nativo; e InfoDispositivo demuestra una tercera implementación genuinamente diferente. Android fue compilado, probado y ejecutado. El código Kotlin iOS compila, pero su ejecución queda fuera del alcance actual por falta del entorno Apple.